

Architectural Redesign

Movie Explorer System

The initial three-tier architecture of the Movie Explorer system provided a solid foundation with a React frontend, Node.js/Express backend, and PostgreSQL database. After further analysis, several minor refinements have been made to strengthen separation of concerns, improve code organisation, and enhance maintainability without fundamentally altering the original architecture. This section describes these incremental improvements, the patterns applied at the module level, and updated diagrams reflecting the refined structure.

Refined Architectural Approach

- The system continues to follow the three-tier client-server architecture. The core structure (React SPA communicating with an Express REST API backed by PostgreSQL) remains unchanged. What has been refined is the internal organisation of the backend and the introduction of small, focused design patterns within existing modules.
- The original backend had controller functions that sometimes mixed route handling, business logic, and database queries. The refinement introduces a clearer internal separation within the backend by explicitly distinguishing between controllers, services, and data access responsibilities. This is not a new architectural style but rather a tidying of the existing structure to follow established software engineering practices more closely.

Applied Design Patterns (Minimal Additions)

Rather than introducing entirely new layers or complex patterns, the following lightweight improvements have been integrated into the existing architecture:

- Singleton Pattern
 - The TMDB API adapter is now instantiated once and shared across the application. Previously, each request could create a new HTTP client configuration; using a singleton ensures consistency, reduces overhead, and centralises API key management.
 - The database connection pool is configured as a singleton, ensuring a single connection pool handles all queries efficiently.
- Repository Pattern (Implicit Refinement)
 - The original design already placed database queries in a dedicated data access layer. This has been refined so that each module in the data access layer (user queries, watchlist queries, cache queries) is grouped

into clear repository-style files. This improves testability and keeps SQL isolated from business logic.

- **Error Handling Middleware**
 - A single, centralised error-handling middleware function has been added to the Express application. Instead of scattering try-catch blocks in every controller, unhandled errors propagate to this middleware, which formats a consistent JSON error response. This was a **missing piece** in the original design and improves reliability.
- **Input Validation at Controller Level**
 - A lightweight validation step is now performed at the beginning of each controller function before passing data to the service layer, using a simple validation utility. This prevents malformed or malicious data from reaching deeper layers without introducing a heavy validation library.

Updated Main Components and Responsibilities

No new components have been added; existing ones have been slightly reorganised.

- **Frontend (React)**
 - **Pages:** SearchPage, MovieDetailPage, DashboardPage, LoginPage, RegisterPage – each mapped to a route via React Router.
 - **Reusable components:** SearchBar, MovieCard, MovieGrid, FilterPanel, Navbar, ProtectedRoute.
 - **API client (Axios instance):** Centralised HTTP configuration with an interceptor that attaches the JWT token automatically.
- **Backend (Node.js + Express)**

The backend retains its original modules, now organised more clearly:

- **Controller Layer (previously: route handlers):** Controllers parse incoming requests, perform lightweight validation, call the appropriate service, and send HTTP responses. They contain no business logic.
- **Service Layer:** Contains business logic. MovieService orchestrates fetching from cache or TMDB; UserService handles authentication logic; WatchlistService enforces rules like preventing duplicate additions.
- **Data Access Layer (refined repository grouping):** Database queries are grouped by domain: userQueries, watchlistQueries, movieCacheQueries. Each exports clean functions like findUserByEmail, insertWatchlistItem, getCachedMovie.
- **Infrastructure / Utilities:**
 - **tmdbAdapter (singleton):** Handles all communication with the external TMDB API.

- authMiddleware: Verifies JWT tokens on protected routes.
 - errorHandler: Catches and formats all unhandled errors.
- **Database (PostgreSQL):** Tables: users, watchlists, favourites, cached_movies. Schema remains unchanged.

RESTful API Structure (Refined)

The API endpoints remain the same as in the initial design. The only change is that all endpoints now return a standardised response envelope:

```
{  
  "data": { ... },  
  "message": "Operation successful"  
}
```

And for errors:

```
{  
  "error": {  
    "code": "MOVIE_NOT_FOUND",  
    "message": "No movie found with the given ID"  
  }  
}
```

This consistency helps the frontend handle responses predictably.

Data Flow (Minor Refinements)

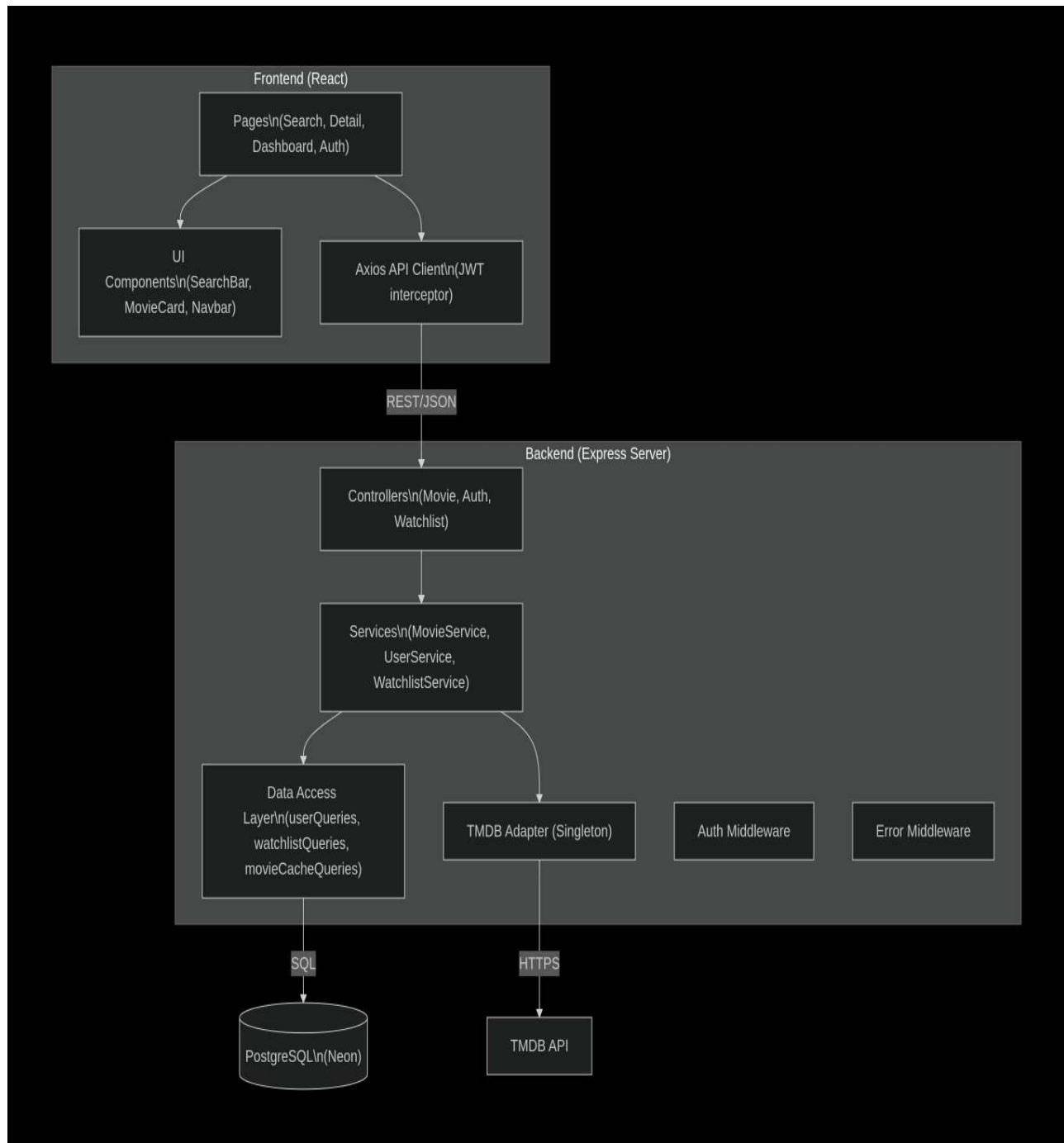
The data flow follows the same path as the original design, with two small improvements:

1. Requests arriving at a controller are validated before reaching the service layer.
2. If any error occurs in the service or data access layers and is not explicitly handled, it propagates to the central error middleware, ensuring the user always receives a structured error response.

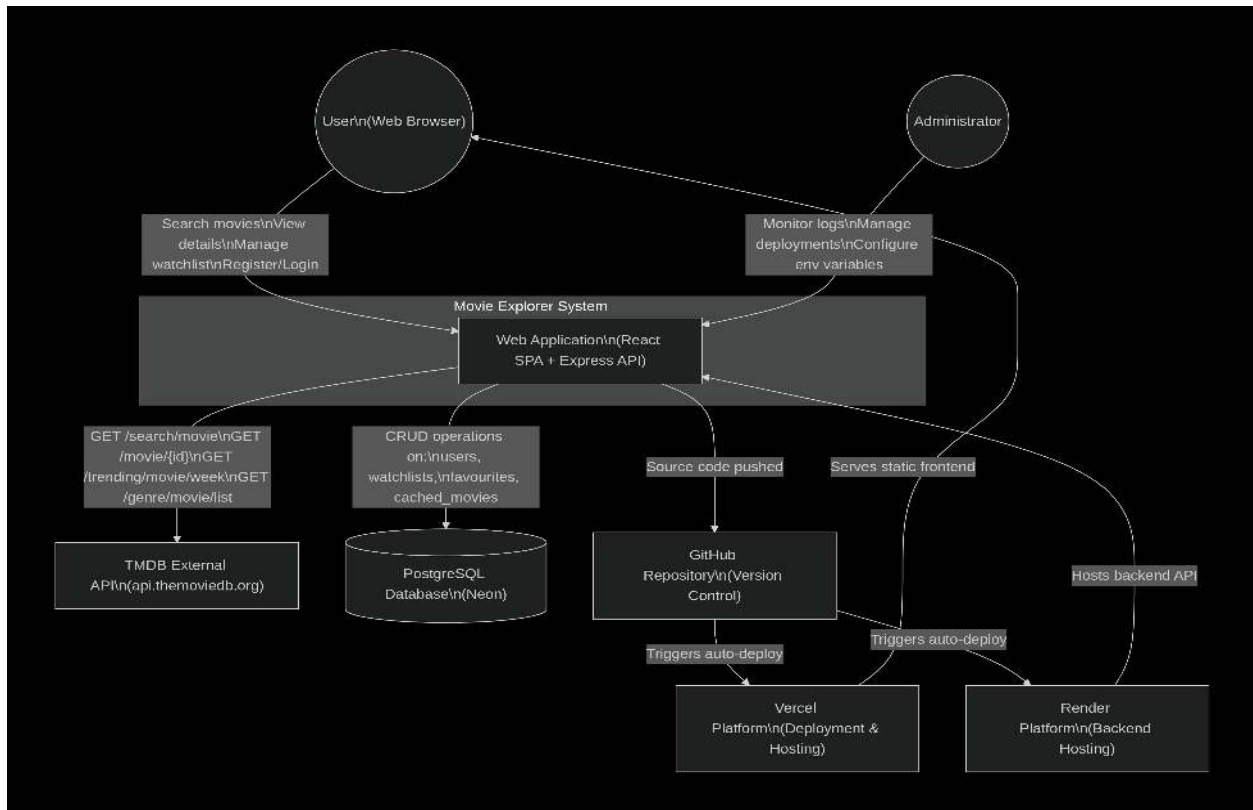
The core flow remains: **User → Frontend → Controller → Service → Data Access / TMDB Adapter → Response → Frontend → User.**

Updated Diagrams

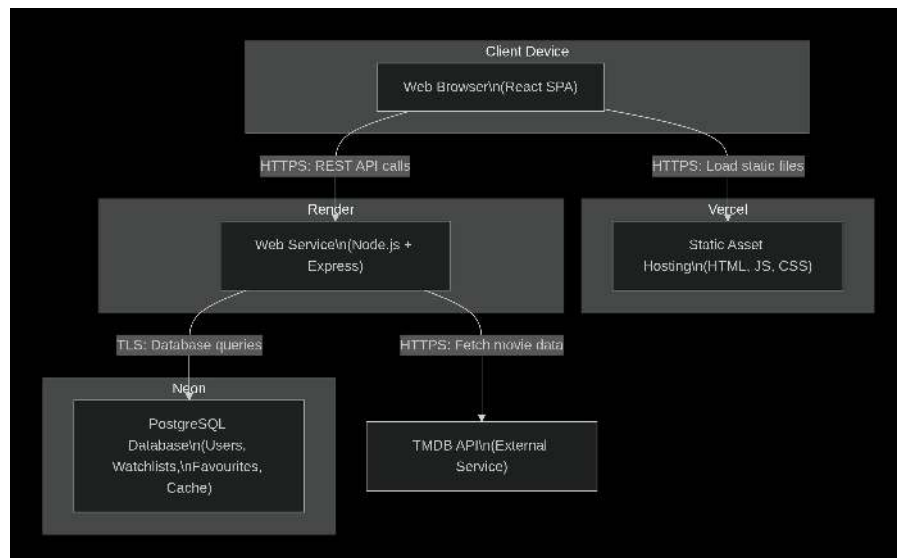
Component Diagram (Refined)



Context Diagram (Expanded: More External entities and Data Flows)



Deployment Diagram (Unchanged)



These refinements preserve the original architecture while addressing small gaps that could have led to **code duplication**, **inconsistency**, or **fragility** as the project grew.